



React for Beginners — Assignment 1

Your first React workout · covers the “React for Beginners” slide deck

ESTIMATED TIME

2–3 hours

DIFFICULTY

Beginner — gentle

TOOLS

Node.js + Vite + your editor

TOTAL POINTS

50 (+5 bonus)

Before you start: keep the slide deck open next to you — every question here maps to a slide. It is *not* cheating to look things up; that is how real developers work. Do the parts in order: they get more hands-on as you go.

Part A · Warm-up: quick concepts

10 pts · 2 each

Answer in one or two sentences. Short and simple beats long and vague.

1. In your own words: what is a **component** in React?

2. This JSX is broken: `<div class="card">` — what is wrong, and what is the fix?

3. **Props** vs **state**: which one can a component change itself, and which one is read-only?

4. What does the empty array do in `useEffect(() => { ... }, [])`?

5. Why does React need a `key` prop when you render a list with `.map()`?

Part B · Bug hunt

15 pts · 5 each

Each snippet contains **one classic beginner bug** from the “classic mistakes” slide. Circle or underline the bug, then write the corrected line underneath.

BUG 1 The alert fires as soon as the page loads — not when the button is clicked.

```
function SaveButton() {  
  function handleSave() {  
    alert('Saved!');  
  }  
  
  return <button onClick={handleSave()}>Save</button>;  
}
```

BUG 2 Clicking “Add” updates the array — but the screen never changes.

```
function TodoApp() {  
  const [todos, setTodos] = useState(['Learn JSX']);  
  
  function addTodo() {  
    todos.push('New task'); // hmm...  
    setTodos(todos);  
  }  
  
  return <button onClick={addTodo}>Add</button>;  
}
```

BUG 3 The list renders, but the console shows a warning on every refresh.

```
const skills = [  
  { id: 1, name: 'HTML' },  
  { id: 2, name: 'CSS' },  
  { id: 3, name: 'JavaScript' },  
];  
  
function SkilllList() {  
  return (  
    <ul>  
      {skills.map(skill => (  
        <li>{skill.name}</li>  
      ))}  
    </ul>  
  );  
}
```

Part C · Build: “My Mini Profile”

25 pts · 5 each

A tiny app about *you*. Each step uses exactly one concept from the slides — if a step feels hard, find its slide and copy the pattern. Start by scaffolding a project:

```
npm create vite@latest my-profile -- --template react
cd my-profile
npm install
npm run dev
```

- 1 **A ProfileCard component with props**
Create `ProfileCard.jsx`. It receives `name` and `bio` as props and displays them. Render it from `App.jsx`, passing your own name and a one-line bio.
- 2 **A Like button with state**
Add a `♥ Like` button to the card using `useState(0)`. Every click increases the number: “12 likes”. Remember: pass the function, don’t call it.
- 3 **A skills list with keys**
Make an array of at least four skills (objects with `id` and `name`). Render them as a list with `.map()` — no key warnings allowed in the console.
- 4 **A “Show more” toggle with conditional rendering**
Add a second, longer bio paragraph that only appears when a `showMore` state is true. One button toggles it, and its label flips between “Show more” and “Show less” (use a ternary).
- 5 **A status message with a controlled input**
Add an `<input>` above the card wired to state with `value` + `onChange`. Whatever you type appears live inside the card as your status, e.g. “Learning React 🚀”.

★ **Bonus (+5):** use `useEffect` to keep the browser tab title in sync with the card — the tab should read “Ada — 12 likes” and update on every like (set `document.title`). Think carefully about what belongs in the dependency array.

Wrap-up · Checklist & grading

before you submit

- ☐ The app runs with `npm run dev`
- ☐ The browser console shows no errors or warnings
- ☐ `ProfileCard` lives in its own file
- ☐ Props are destructured in the function signature
- ☐ State is never mutated directly — no `.push()`
- ☐ Part A & B answers are written on this sheet

SECTION	POINTS
Part A — Warm-up concepts	10
Part B — Bug hunt	15
Part C — My Mini Profile	25
★ Bonus — <code>useEffect</code> title	+5
TOTAL	50 (+5)